Data Science Practical

Department of Statistics
Ludwig-Maximilians-Universität München

Simon Drauz, Zoe Matrullo

Submitted in partial fulfillment of the requirements for the degree of M. Sc.
Supervised by Prof. Dr. Matthias Schubert, Ajay Menon

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Contents

1 Introduction

Trajectory prediction models are almost universally evaluated through aggregate error metrics computed over an entire test set, most commonly the Average Displacement Error (ADE) and the Final Displacement Error (FDE). While such summary statistics are useful for ranking models against one another, they provide a fundamentally incomplete picture of model behaviour. A single averaged number reveals neither the conditions under which a model succeeds nor those under which it fails. Two models with identical mean ADE may behave very differently in practice: one might perform uniformly across all situations, while another might excel on common, easy cases and fail catastrophically on rarer, safety-critical ones.

This limitation is particularly consequential in the autonomous driving context. Some trajectories are inherently hard to predict, such as those of a pedestrian who abruptly changes direction or navigates a densely crowded crossing, whereas others are close to trivial, such as an agent walking in a straight line at constant speed across an empty scene. An aggregate metric collapses this heterogeneity into a single value and offers no insight into *why* certain predictions fail and others succeed. Consequently, the standard evaluation paradigm provides no understanding of which trajectory characteristics or scene properties drive the per-trajectory prediction error of a given model.

This work addresses that gap by developing a comprehensive pipeline to understand the drivers of prediction error. Rather than proposing a new prediction architecture or attempting to improve aggregate accuracy, we build an interpretable analysis pipeline that decomposes Trajectron++ prediction errors into their constituent factors. The pipeline combines per-trajectory error recovery, feature engineering, and interpretable meta-models to characterise which trajectory and scene characteristics, as well as which model configurations, systematically influence prediction performance.

We structure the investigation around three research questions:

RQ1 — Which trajectory and scene characteristics affect model performance? We seek to identify the features, such as speed, speed variability, acceleration, heading change, and scene density, that systematically influence the per-trajectory prediction error, and to quantify the direction and magnitude of their effects.

RQ2 — What are the success and failure modes that explain model performance? Beyond the marginal effect of individual features, we ask whether trajectories cluster into coherent regimes that the model consistently handles well or poorly, and whether these regimes can be characterised in human-interpretable terms.

RQ3 — How do model settings affect performance? Finally, we ask how Trajectron++'s own configuration, specifically the observation history length, the prediction horizon, and the attention radius, influences prediction quality, independently of the intrinsic difficulty of each trajectory.

These three questions map directly onto the structure of the remainder of this report. RQ1 and RQ2 are addressed through an interpretable meta-modelling and clustering pipeline applied to a fixed model configuration, while RQ3 is addressed through a controlled sweep over model settings.

2 Background and Related Work

2.1 Trajectory Prediction

Trajectory prediction is the task of forecasting the future positions of agents such as pedestrians, cyclists, or vehicles, given a sequence of their observed past movements. It is a foundational problem in autonomous driving, robotics, and smart city applications, where anticipating agent behaviour is critical for safe and efficient operation.

Early approaches relied on physics-based models such as the Social Force Model (Helbing and Molnar, 1995), which simulates pedestrian motion as a system of attractive and repulsive forces. While interpretable and computationally lightweight, these models struggle to capture the complexity and variability of real-world behaviour.

The introduction of recurrent neural networks marked a significant shift. Social-LSTM (Alahi et al., 2016) extended the LSTM architecture with a social pooling mechanism that allows agents to share hidden states with their neighbours, enabling the model to account for crowd interactions. Social-GAN (Gupta et al., 2018) built on this by framing prediction as a generative task using a generative adversarial network, producing a diverse set of socially plausible future trajectories rather than a single deterministic output.

More recent work has embraced graph-based and attention-based architectures to better capture structured interactions. DESIRE (Lee et al., 2017) and Trajectron++ (Salzmann et al., 2020) combine probabilistic reasoning with multi-agent interaction modelling. Trajectron++ in particular offers a principled framework for heterogeneous, multi-agent trajectory prediction: it represents the scene as a dynamic graph, uses recurrent encoders to model each agent's history, and employs conditional variational autoencoders (CVAEs) to produce a distribution over future trajectories. Importantly, Trajectron++ supports integration of map and semantic context, and accommodates multiple agent types including pedestrians, cars, and cyclists, making it well-suited to complex real-world datasets.

Transformer-based models such as AgentFormer (Yuan et al., 2021) have further advanced the field by jointly modelling spatial and temporal dependencies across agents using self-attention, avoiding the sequential bottleneck of recurrent networks. Diffusion-based approaches such as MID (Gu et al., 2022) have also been explored, applying denoising diffusion probabilistic models to generate diverse and accurate multimodal futures.

2.2 The nuScenes Dataset

nuScenes (Caesar et al., 2020) is a large-scale, multimodal autonomous driving dataset collected in Boston and Singapore. It provides 1000 driving scenes, each 20 seconds long, with 3D bounding box annotations for 23 object categories at 2 Hz. The dataset includes data from six cameras, a 32-beam LiDAR, and five radars, enabling rich multi-sensor perception. Each annotated object is assigned a semantic category such as car, pedestrian, or bicycle, as well as an attribute such as moving, stopped, or parked.

For trajectory prediction, nuScenes offers a standardised benchmark with defined observation and prediction horizons, typically 2 seconds of history and 6 seconds of future prediction, along with an official set of evaluation metrics. The dataset's diversity across two cities, varying traffic densities, and heterogeneous agent types makes it a challenging

and realistic testbed. At the same time, this heterogeneity means that prediction difficulty varies substantially across scenes and agent types, motivating a deeper investigation of which trajectory characteristics drive model performance.

2.3 Evaluation Metrics for Trajectory Prediction

Trajectory prediction is most commonly evaluated using displacement-based metrics. The Average Displacement Error (ADE) measures the mean Euclidean distance between predicted and ground-truth positions across all future timesteps. The Final Displacement Error (FDE) measures this distance only at the final predicted timestep, capturing whether the model correctly anticipates the agent's eventual position.
Since modern models are generative and produce multiple hypotheses, metrics are often computed under a best-of-k protocol: k samples are drawn from the model, and the minimum error across samples is reported as ADE-k and FDE-k. This rewards a model for including the correct future within its predictive distribution, without penalising it for also generating other plausible alternatives. Common values of k are 5 and 10.
Beyond aggregate statistics, recent work has highlighted the importance of disaggregated evaluation. Performance can vary significantly across agent types, motion patterns such as straight-line versus turning trajectories, scene density, and speed regimes. Understanding these sources of variation is important both for characterising model limitations and for guiding further development.

2.4 Interpretable Machine Learning for Model Analysis

When the goal is not to maximise predictive accuracy but to understand what drives a model's performance, interpretable machine learning methods offer a valuable toolkit. Rather than treating a complex model as a black box, these methods build transparent secondary models that relate input features to an outcome of interest, in this case prediction error.
Generalised Additive Models (GAMs) (Hastie and Tibshirani, 1990) decompose the prediction into a sum of smooth univariate functions, one per feature, making it possible to visualise each feature's marginal contribution to the outcome. Explainable Boosting Machines (EBMs) (Lou et al., 2013, Nori et al., 2019) extend this framework using gradient boosting to learn these shape functions and additionally capture pairwise interaction terms, achieving accuracy competitive with more complex models while retaining full interpretability. Gradient Boosted Trees (Friedman, 2001), as implemented in libraries such as XGBoost or LightGBM, sacrifice some interpretability for higher flexibility; feature importance scores and SHAP values (Lundberg and Lee, 2017) can then be used to recover post-hoc explanations.
In the context of trajectory prediction analysis, these methods allow one to formally quantify how trajectory-level features such as agent speed, trajectory curvature, observation length, or scene density relate to the prediction error assigned by Trajectron++. This provides a principled, data-driven characterisation of the model's strengths and failure modes, going beyond anecdotal inspection of individual examples.

2.5 Federated Learning and Personalisation

Federated learning (McMahan et al., 2017) is a distributed training paradigm in which multiple clients such as vehicles or edge cameras collaboratively train a shared model without exchanging raw data. Each client trains locally and shares only model updates with a central server, which aggregates them into an updated global model via algorithms such as FedAvg. This preserves data privacy and reduces communication overhead.
A key challenge in federated learning arises when client data distributions are heterogeneous, a setting commonly referred to as non-IID. In the trajectory prediction setting, this corresponds to clients located in environments with structurally different movement patterns, such as a highway versus a pedestrian plaza. A single global model may perform well on average but poorly for individual clients whose data deviates from the mean. Personalised federated learning addresses this by allowing clients to maintain partially or fully individualised models. Strategies include split-model approaches such as FedPer and FedRep (Arivazhagan et al., 2019, Collins et al., 2021), which share a common encoder while giving each client its own prediction head; regularisation-based approaches such as pFedMe (Dinh et al., 2020) and DITTO (Li et al., 2021), which penalise deviation from the global model during local training; and meta-learning approaches such as Per-FedAvg (Fallah et al., 2020), which optimise for rapid local adaptation with few gradient steps.
While the present work focuses on a centralised, in-depth analysis of Trajectron++ rather than a federated deployment, understanding the factors that govern prediction quality across diverse trajectory types is directly relevant to motivating and designing personalisation strategies in future federated settings.

3 Methodology

3.1 Trajectron++ Adaptation for Per-Trajectory Evaluation

Answering the research questions posed in Section 1 requires prediction errors at the granularity of individual trajectories. The standard Trajectron++ evaluation procedure does not expose this information, and overcoming this limitation is the first methodological contribution of our pipeline.

3.2 Limitations of the Standard Evaluation Loop

In its original form, the Trajectron++ evaluation loop reports batch-averaged ADE and FDE values. Predictions are generated for each batch of agents, the displacement errors are computed, and these errors are immediately aggregated into summary statistics. The intermediate, per-agent errors are not retained, and crucially there is no mechanism to associate a given error value with the specific trajectory that produced it.
This design is entirely adequate for the model's original purpose, namely benchmarking aggregate accuracy against competing approaches. However, it is incompatible with our analysis. Because the per-trajectory errors are discarded and never linked to an identifier, it is impossible to study what makes a particular trajectory hard or easy: there is no way to relate the prediction error of a trajectory to its kinematic properties or to the characteristics of the scene in which it occurred.

3.3 Recovering Individual Trajectory Errors

To enable per-trajectory analysis, we modified the Trajectron++ evaluation loop so that it preserves the individual prediction loss for every trajectory rather than only the batch average. Concretely, instead of reducing the displacement errors to a mean within each batch, we retain the full vector of per-trajectory errors together with the metadata required to identify each trajectory.

The central enabling component of this modification is the use of a stable trajectory identifier provided by the `trajdata` library, which we refer to throughout as the `data_idx`. This identifier uniquely and persistently labels each trajectory sample across the data-loading and evaluation stages. Because the same identifier is available both when the model produces its prediction error and when we compute the trajectory and scene features (described in Section 3.5), the `data_idx` acts as a join key. It allows us to construct, for every trajectory, a single record that pairs the Trajectron++ prediction error with the full set of characteristics describing that trajectory and its surrounding scene.

Without this stable join key, the two halves of the analysis, the prediction errors on one side and the interpretable features on the other, could not be reliably aligned, and the entire downstream meta-modelling effort would be impossible. The modification is conceptually simple but is the linchpin that converts Trajectron++ from a black-box benchmark into a source of structured, per-trajectory observations suitable for interpretable analysis.

3.4 Scope of the Prediction Task

Consistent with the dataset scope described in Section **??**, we focus the prediction and evaluation on pedestrian trajectories, while all other agent types remain present in each scene and contribute to the model's interaction reasoning as context. The per-trajectory errors recovered through the modified evaluation loop therefore correspond to pedestrian targets, and it is these errors that serve as the response variable for the interpretable models in the subsequent sections.

3.5 Feature Engineering and Data Preparation

With per-trajectory prediction errors available, the next stage of the pipeline constructs the set of interpretable features that will be related to those errors. A key point is that Trajectron++ itself outputs only sequences of (x, y) positions together with their associated time stamps; it does not expose any higher-level descriptors of motion or scene structure. Every feature described below is therefore computed by us directly from the raw trajectory coordinates and the scene context, not taken from the model.

3.6 Trajectory and Scene Characteristics

We organise the engineered features into two groups: agent motion features, which describe the kinematics of the focal trajectory, and scene context features, which describe the environment in which the trajectory unfolds.

Agent motion features. These capture the dynamics and geometric complexity of the focal agent's movement. They include:

- **Speed** statistics: the mean, maximum, and standard deviation of speed over the trajectory. The standard deviation in particular captures how much the agent's speed fluctuates, which is a proxy for irregular or unpredictable motion.
- **Acceleration** statistics: the mean and maximum acceleration, describing how rapidly the agent changes speed.
- **Jerk** statistics: the mean and maximum jerk, that is, the rate of change of acceleration, which captures the smoothness of the motion.
- **Heading change per second**, quantifying how much the agent turns over time and thus distinguishing straight-line motion from curved or erratic paths.
- **Minimum neighbour distance**, the closest approach to any other agent, which reflects how tightly the agent must navigate around others.

Scene context features. These describe the broader environment and the degree of crowding around the focal agent. They include the number of agents in the scene, the number of vehicles specifically, the scene bounding-box area, and the spatial agent density. Taken together, these two groups yield 18 candidate features. Each feature vector is associated with its corresponding Trajectron++ prediction error through the `data_idx` join key introduced in Section 3.1, producing the unified per-trajectory dataset on which all subsequent modelling is performed.

3.7 Target Distribution and Log Transformation

The response variable for the interpretable models is the per-trajectory most-likely ADE, denoted `ml_ade`. Before fitting any model, we examined the distribution of this target. The raw `ml_ade` distribution is strongly right-skewed, with a skewness of approximately 2.59: the majority of trajectories incur low prediction error, but a long tail of high-error cases extends far to the right.

Such pronounced skewness is problematic for regression models that assume approximately symmetric, homoscedastic residuals, as is the case for the additive model discussed in Section 3.9. To address this, we considered a $\log(1 + x)$ (`log1p`) transformation of the target, which compresses the long upper tail and yields a substantially more symmetric distribution. Whether to model the error on the original scale or the log scale was treated as an explicit modelling choice that we evaluated in the subsequent stage rather than fixing a priori, so that the decision could be made on the basis of model fit and diagnostic behaviour.

3.8 Feature Selection

With 18 candidate features, many of which are correlated by construction (for example, the various speed statistics), an unfiltered feature set would introduce substantial multicollinearity. Multicollinearity is especially damaging for interpretability: when features

are highly correlated, the estimated effect of any one of them becomes unstable and difficult to attribute, undermining the very goal of the analysis. We therefore performed feature selection, and to ensure that our conclusions were not an artefact of a single selection strategy, we implemented and compared two complementary approaches.

Approach 1: Mutual-information elbow followed by VIF. We first ranked all 18 candidate features by their mutual information (MI) with the target, which measures the strength of the (possibly non-linear) statistical dependence between each feature and the prediction error. Inspecting the ranked MI curve revealed an elbow, and we discarded the low-signal features lying below it, reducing the set from 18 to 10. We then applied a variance inflation factor (VIF) filter with a threshold of 5 to remove residual multicollinearity, which further reduced the set to 4 features: `std_speed`, `mean_acceleration`, `max_speed`, and `heading_change_per_sec`. Notably, this procedure retains only motion features; all scene-context features fall below the MI elbow, indicating that, considered individually, they carry comparatively little information about per-trajectory prediction error. All four retained features are highly statistically significant ($p \ll 0.01$).

Approach 2: VIF only. As a contrasting strategy, we skipped the MI ranking entirely and applied the VIF filter (again with a threshold of 5) directly to all 18 candidates. This yielded 6 features: `std_speed`, `mean_acceleration`, `min_neighbor_distance`, `heading_change_per_sec`, `scene_num_VEHICLE`, and `scene_density_VEHICLE`. In contrast to the first approach, this set retains several scene-level features alongside the motion features.

Comparing the two feature sets serves as a robustness check. The MI-elbow-plus-VIF set is more parsimonious and motion-focused, whereas the VIF-only set deliberately preserves scene-context information that the MI filter would otherwise remove. Carrying both forward allows us to assess whether the inclusion of scene features materially changes the conclusions of the interpretable analysis, an assessment we return to in the results.

3.9 Interpretable Models: GAM and XGBoost

The per-trajectory `ml_ade` values, together with the selected features, allow us to ask not merely *how large* the prediction error is for a given trajectory, but *why* it is large or small. The error value alone is uninformative in this respect: it tells us that variation exists but not what drives it. To uncover the drivers, we fit a second-stage, interpretable model, a *meta-model*, that takes the engineered trajectory and scene features as inputs and predicts the Trajectron++ prediction error as output. If this meta-model is itself interpretable, then its learned structure can be read as an explanation of the conditions under which Trajectron++ performs well or poorly.

We adopt two complementary meta-models: a Generalized Additive Model (GAM), which is inherently interpretable, and XGBoost, a gradient-boosted tree ensemble that is more flexible but must be interpreted post-hoc. Using both in tandem provides a valuable cross-check: where the transparent additive model and the flexible ensemble agree on which features matter and in what direction, we can place substantially greater confidence in the conclusion.

3.10 Generalized Additive Model

A Generalized Additive Model expresses the target as a sum of smooth, univariate functions of the individual features:

$$f(x) = \sum_j s_j(x_j), \tag{1}$$

where each s_j is a smooth spline learned from the data. The defining property of this formulation is that the contribution of each feature is captured by its own dedicated function and these contributions are simply added together. There are no interaction terms: the model assumes that the effect of one feature does not depend on the value of another.
This additive structure makes the GAM inherently interpretable. Each s_j can be plotted directly to reveal the marginal effect of feature x_j on the predicted error, showing immediately whether the relationship is increasing, decreasing, linear, or non-linear. Moreover, the GAM framework provides p-values for each smooth term, enabling formal significance testing of whether a feature has a statistically detectable effect on prediction error. The interpretability of the GAM is thus structural: the explanation is the model, requiring no additional attribution machinery.

3.11 XGBoost

XGBoost (Chen and Guestrin, 2016) is an ensemble method based on gradient-boosted decision trees. Rather than fitting a single model, it constructs a sequence of decision trees in which each new tree is trained to correct the residual errors of the trees already in the ensemble. The prediction for an input x after T boosting rounds is the sum of the outputs of all the individual trees:

$$\hat{y}(x) = \sum_{t=1}^{T} f_t(x), \tag{2}$$

where each f_t is a regression tree. Each tree partitions the feature space through a sequence of threshold rules and assigns a constant correction to each resulting region; because a single tree can split on several features in succession, the ensemble naturally captures non-linearities and, importantly, interactions between features. This flexibility is precisely what the additive GAM lacks, and it allows XGBoost to represent effects such as "high speed variability only increases error when it is accompanied by sharp turning".
The cost of this flexibility is that the fitted ensemble, typically comprising many trees, is not directly human-readable. We therefore interpret it post-hoc using two standard techniques: SHAP values and partial dependence plots. SHAP values decompose each individual prediction into additive, signed contributions from each feature on top of a baseline value:

$$f(x) = \phi_0 + \sum_j \phi_j(x), \tag{3}$$

where ϕ_0 is the baseline (the mean model prediction) and each $\phi_j(x)$ quantifies how much feature j pushes the prediction for this specific input above or below that baseline. Aggregating the magnitude of these contributions across the dataset yields a global measure of

feature importance, while partial dependence plots visualise the average effect of a feature across its range, analogous in spirit to the smooth terms of the GAM but recovered from a black-box model.

3.12 Rationale for Using Both Models

The two meta-models occupy complementary positions on the interpretability–flexibility spectrum. The GAM is fully transparent and provides significance tests, but its additive assumption blinds it to interactions. XGBoost can capture those interactions and typically attains a better fit, but requires post-hoc tools to be interpreted and offers no native significance testing. Reading the two together mitigates the weaknesses of each: agreement between a transparent additive model and a flexible interaction-aware ensemble is strong evidence that an identified effect is real rather than an artefact of one model's assumptions. This dual-model strategy underpins the feature-importance and feature-effect analyses that answer RQ1 and RQ2.

3.13 Model Setting Sweep

The analysis described so far holds the Trajectron++ configuration fixed and studies how trajectory and scene characteristics relate to prediction error. RQ3 turns to a complementary question: how does the model's own configuration affect prediction quality? To answer this, we conducted a controlled sweep over three of Trajectron++'s key settings and analysed their effect using the same interpretable meta-modelling framework developed in Section 3.9.

3.14 Sweep Design

We swept over three model settings:

- `history_sec` — the length of the observed history window, that is, how much of each agent's past motion the model is given as input.
- `prediction_sec` — the prediction horizon, that is, how far into the future the model forecasts.
- `attention_radius_m` — the attention radius, the distance in metres within which neighbouring agents are included in the model's interaction graph; agents beyond this radius are excluded from the focal agent's neighbourhood.

We performed a Cartesian sweep over these three parameters, evaluating the model at every combination of the swept values. The sweep was carried out on the nuScenes mini split rather than the full dataset, because the repeated retraining and evaluation required by a full Cartesian sweep is computationally prohibitive on the complete dataset. The mini split nonetheless provides sufficient data to detect the principal effects of each setting.

3.15 Pooled Model and Confound Control

To isolate the effect of the model settings from the intrinsic difficulty of individual trajectories, we fit a single *pooled* meta-model that includes both the model-setting variables and the trajectory and scene features as predictors. The trajectory and scene features act as control variables: by accounting for the fact that different trajectories are inherently more or less difficult, the pooled model attributes the residual variation in error to the settings themselves. This design allows the effect of, say, the prediction horizon to be estimated while holding trajectory difficulty constant.
A subtle but important issue in this setup is mechanical confounding between the settings and certain trajectory features. Some features scale trivially with a setting: for instance, the total path length of a trajectory grows mechanically with the prediction horizon, simply because a longer horizon covers more time, irrespective of any change in the agent's behaviour or the difficulty of the prediction. If left uncorrected, such features would absorb part of the genuine effect of the setting and distort the analysis. To prevent this, we used setting-agnostic, normalised versions of the affected features, for example `mean_path_length_per_second` in place of the raw `path_length`. This normalisation ensures that the estimated effects of the model settings reflect genuine changes in prediction difficulty rather than arithmetic artefacts of the sweep.

4 Results

4.1 Feature Effects on Prediction Error

Speed variability (`std_speed`) and turning (`heading_change_per_sec`) emerge as the dominant drivers of per-trajectory error, and they do so consistently across both the GAM and the XGBoost meta-model. Maximum speed (`max_speed`) is a secondary contributor, whereas mean acceleration has a near-zero effect. A clear and somewhat unexpected result is that scene-context features such as agent density and vehicle count carry little explanatory power for per-trajectory error: the difficulty of a prediction is governed far more by the focal agent's own motion than by the crowdedness of its surroundings.

4.2 Success and Failure Modes

The feature effects resolve into coherent behavioural regimes. The model succeeds on two broad classes of trajectory: slow turners, where low speed bounds the impact of turning, and straight walkers, where stable heading bounds the impact of speed. Conversely, the failure mode is erratic motion in which speed variability and turning occur together. Crucially, raw speed alone is not a failure driver; high speed becomes problematic only when paired with turning. The hardest cases for Trajectron++ are thus those that combine the two dominant difficulty factors simultaneously.

4.3 Feature Importance of Model Settings

Applying the pooled meta-model of Section 3.13, we quantified the influence of each model setting on prediction error using mean absolute SHAP values from the XGBoost model,

Table 1: Mean absolute SHAP values from the pooled XGBoost model, ranking the influence of model settings and trajectory features on per-trajectory prediction error. Larger values indicate greater influence.

Feature	**Mean** |SHAP|
`prediction_sec`	**0.178**
`attention_radius_m`	0.098
`std_speed`	0.093
`max_speed`	0.070
`heading_change_per_sec`	0.039
`min_neighbor_distance`	0.029
`history_sec`	0.022
`mean_acceleration`	0.013

cross-validated against the GAM. Table 1 reports the resulting global feature importance. The most striking observation is that the two highest-ranked features are model settings rather than trajectory characteristics. The prediction horizon, `prediction_sec`, is by a clear margin the most influential feature overall, with a mean absolute SHAP value of 0.178, and the attention radius, `attention_radius_m`, ranks second at 0.098. The trajectory features, led by `std_speed` at 0.093, remain significant contributors but are surpassed by these two settings. In other words, within the swept ranges, the choice of configuration influences prediction quality at least as strongly as the intrinsic properties of the trajectories being predicted.

All eight features in the pooled model are also statistically significant in the GAM ($p \ll 0.01$), confirming that the effects are not specific to the tree-based model. The two meta-models do, however, differ markedly in fit quality: XGBoost achieves $R^2 = 0.90$ with an RMSE of 0.13, substantially better than the GAM's $R^2 = 0.65$ and RMSE of 0.24. This gap is itself informative. Because the GAM is purely additive while XGBoost can represent interactions, the superior fit of the latter indicates that meaningful interaction effects exist between the model settings and the trajectory features, effects that an additive model cannot capture.

We now examine each setting in turn through its partial dependence plot (PDP), which shows the average direction and shape of its effect on the log-scale prediction error.

4.4 Prediction Horizon

The prediction horizon exhibits the largest effect of any feature. Its partial dependence plot rises monotonically: as the horizon lengthens, the prediction error grows. This is intuitively unsurprising, since forecasting further into the future is inherently harder and accumulated uncertainty compounds over time. The value of the analysis lies in quantifying this effect and establishing that, within our experimental range, the horizon setting exerts a stronger influence on prediction quality than any intrinsic trajectory characteristic.

4.5 Attention Radius

The attention radius is the second most influential feature, and its effect is more nuanced than that of the horizon. A larger radius admits more neighbouring agents into the focal agent's interaction graph, supplying additional context. One might expect more context to be uniformly beneficial, but the partial dependence plot reveals a non-linear relationship. At very small radii, performance is poor because the model omits genuinely relevant neighbours. Beyond a certain point, however, enlarging the radius begins to introduce distant, irrelevant agents whose inclusion adds noise rather than signal. The plot indicates that a comparatively short attention radius is beneficial in this dataset, with performance degrading as the radius becomes large. This points to the existence of an intermediate optimal range rather than a "larger-is-better" rule.

4.6 History Window

The history window has the smallest effect of the three settings. Its partial dependence plot is comparatively flat across the swept range, indicating that the model is robust to the amount of past motion it observes, at least within the values we tested. From a practical standpoint, this is a reassuring finding: it suggests that Trajectron++ does not depend critically on a long observation window to produce reasonable predictions, and that the history length can be chosen with relatively little concern for its impact on accuracy within this regime.

5 Conclusion and Discussion

This work set out to look beyond aggregate error metrics and ask *why* some trajectories are harder for Trajectron++ to predict than others. By recovering per-trajectory prediction errors, relating them to interpretable trajectory and scene features through GAM and XGBoost meta-models, and conducting a controlled sweep over model settings, we have characterised the model's competence in a fine-grained and actionable way.
The results reveal that trajectory characteristics, particularly speed variability and turning, are dominant drivers of prediction difficulty. Model configuration also plays a critical role, with the prediction horizon and attention radius exerting effects comparable to the trajectory features themselves. These findings provide concrete guidance for understanding and improving trajectory prediction performance.

5.1 Limitations

Several limitations should temper the interpretation of these findings. First, our meta-models explain Trajectron++, not ground-truth difficulty. The identified drivers are inferred through the meta-models as proxies, since Trajectron++ does not expose its internal reasoning directly; the effects we report are therefore model-specific and may not generalise to other prediction architectures.
Second, the meta-model under-predicts at high ADE. Out-of-fold diagnostics show strong agreement between predicted and actual error at low-to-moderate ADE but systematic

under-prediction in the high-error tail. The implication is that our trajectory and scene features do not fully capture what makes the worst cases worst: factors such as map and lane geometry, location, or finer-grained social context appear to be missing from the current feature set. This shortfall is the most likely reason that the clustering of hard cases is comparatively weak, as the available feature effects do not differentiate cleanly within the high-error tail.

Third, the scope of the study is bounded. We predict pedestrian trajectories on nuScenes only, so there is no guarantee that the conclusions transfer to other agent types or to other datasets. Moreover, the settings sweep was conducted on the nuScenes mini split for reasons of computational cost, and its results may not transfer directly to full-scale training.

5.2 Future Work

These limitations suggest several directions for future work. A central strength of our approach is that the pipeline is model- and dataset-agnostic by design: the meta-modelling and feature-effect clustering are decoupled from the prediction model and the dataset, and through the `trajdata` interface the same pipeline can be ported to other trajectory prediction models and datasets with little additional effort. This makes broad replication straightforward.

A natural next step is to enrich the input space with map and lane geometry, location information, and finer social context. Such features directly target the under-prediction ceiling identified in the diagnostics and may unlock cleaner structure in the hard-error regime that the current feature set fails to separate. Finally, extending the analysis to vehicle, cyclist, and motorcyclist trajectories would test whether the same drivers of difficulty, speed variability and turning, govern prediction error across agent types, or whether each agent class exhibits its own characteristic failure modes.

A Appendix

B Electronic appendix

Data, code and figures are provided in electronic form.

References

Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Fei-Fei, L. and Savarese, S. (2016). Social lstm: Human trajectory prediction in crowded spaces, *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 961–971.

Arivazhagan, M. G., Aggarwal, V., Singh, A. K. and Choudhary, S. (2019). Federated learning with personalization layers, *arXiv preprint arXiv:1912.00818* .

Caesar, H., Bankiti, V., Lang, A. H., Vora, S., Liong, V. E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G. and Beijbom, O. (2020). nuScenes: A multimodal dataset for autonomous driving, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11621–11631.

Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.

Collins, L., Hassani, H., Mokhtari, A. and Shakkottai, S. (2021). Exploiting shared representations for personalized federated learning, *arXiv preprint arXiv:2102.07078* .

Dinh, C. T., Tran, N. H. and Nguyen, T. D. (2020). Personalized federated learning with Moreau envelopes, *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, pp. 21394–21405.

Fallah, A., Mokhtari, A. and Ozdaglar, A. (2020). Personalized federated learning: A meta-learning approach, *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, pp. 3557–3568.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine, *Annals of Statistics* **29**(5): 1189–1232.

Gu, T., Yang, J. and Zhang, C. (2022). Motion indeterminacy diffusion for trajectory forecasting, *arXiv preprint arXiv:2203.13777* .

Gupta, A., Johnson, J., Fei-Fei, L., Savarese, S. and Alahi, A. (2018). Social gan: Socially acceptable trajectories with generative adversarial networks, *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2255–2264.

Hastie, T. J. and Tibshirani, R. J. (1990). *Generalized Additive Models*, Chapman & Hall.

Helbing, D. and Molnar, P. (1995). Social force model for pedestrian dynamics, *Physical review E* **51**(5): 4282.

Lee, N., Choi, W., Vernaza, P., Choy, C. B., Torr, P. H. and Chandraker, M. (2017). Desire: Distant future prediction in dynamic scenes with interacting agents, *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 336–345.

Li, T., Hu, S., Beirami, A. and Smith, V. (2021). Ditto: Fair and robust federated learning through personalization, *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 6357–6368.

Lou, Y., Caruana, R., Gehrke, J. and Hooker, G. (2013). Accurate intelligible models with pairwise interactions, *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 623–631.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions, *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.

McMahan, H. B., Moore, E., Ramage, D., Hampson, S. and Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data, *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 1273–1282.

Nori, H., Jenkins, S., Koch, P. and Caruana, R. (2019). InterpretML: A unified framework for machine learning interpretability.

Salzmann, T., Ivanovic, B., Chakravarty, P. and Pavone, M. (2020). Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data, *European conference on computer vision*, Springer, pp. 683–700.

Yuan, Y., Weng, X., Ou, Y. and Kitani, K. (2021). AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting, *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9813–9823.

Declaration of authorship

I hereby declare that the report submitted is my own unaided work. All direct or indirect sources used are acknowledged as references. I am aware that the Thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the report as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future Theses submitted. Further rights of reproduction and usage, however, are not granted here. This paper was not previously presented to another examination board and has not been published.

Location, date

Name